

CT60A2600-TIIMI-A

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 jono Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 pSeuraava	5
3.1.2.2 pSolmu	6
3.2 puu Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	6
3.2.2.1 aNimi	6
3.2.2.2 iArvo	6
3.2.2.3 iPituus	7
3.2.2.4 pOikea	7
3.2.2.5 pVasen	7
3.3 RBSolmu Struct Reference	7
3.3.1 Detailed Description	7
3.3.2 Member Data Documentation	8
3.3.2.1 aNimi	8
3.3.2.2 iArvo	8
3.3.2.3 iVariBitti	8
3.3.2.4 pOikea	8
3.3.2.5 pVanhempi	8
3.3.2.6 pVasen	9
3.4 tiedot Struct Reference	9
3.4.1 Detailed Description	9
3.4.2 Member Data Documentation	9
3.4.2.1 aSukunimi	9
3.4.2.2 iYhteensa	9
3.4.2.3 pEdellinen	10
3.4.2.4 pSeuraava	10
4 File Documentation	11
4.1 src/binaaripuu.c File Reference	11
4.2 src/binaaripuu.h File Reference	11
4.2.1 Typedef Documentation	12
4.2.1.1 JONO	13
4.2.1.2 PUU	13

4.2.1.3 RBSOLMU	13
4.2.2 Function Documentation	13
4.2.2.1 binaariHaku()	13
4.2.2.2 kierraOikealle()	14
4.2.2.3 kierraVasemmalle()	14
4.2.2.4 kirjoitaBinaaripuu()	14
4.2.2.5 kirjoitaRB()	15
4.2.2.6 kirjoitaRBTiedostoon()	15
4.2.2.7 kirjoitaTiedostoon()	16
4.2.2.8 korjaaLisays()	16
4.2.2.9 kysyArvo()	17
4.2.2.10 leveyshaku()	17
4.2.2.11 lisaaRBSolmu()	18
4.2.2.12 lisaaSolmu()	19
4.2.2.13 luoPuu()	20
4.2.2.14 luoRBPuu()	21
4.2.2.15 oikeaPuoli()	21
4.2.2.16 poistaSolmu()	22
4.2.2.17 puunPituus()	22
4.2.2.18 suurempiLukuVertailu()	23
4.2.2.19 syvyyshaku()	23
4.2.2.20 tarkistaLoytyykoSyvyyshaulla()	24
4.2.2.21 tasapainoitaPuu()	24
4.2.2.22 vapautaMuistiJono()	25
4.2.2.23 vapautaMuistiPuu()	25
4.2.2.24 vapautaMuistiRB()	26
4.2.2.25 varaaMuistiaJonolle()	26
4.2.2.26 varaaMuistiaPuulle()	26
4.2.2.27 varaaMuistiaRB()	27
4.2.2.28 vasenPuoli()	27
4.3 src/linkitettylista.c File Reference	28
4.3.1 Function Documentation	28
4.3.1.1 kirjoitaTiedostoAlustaLoppuun()	29
4.3.1.2 kirjoitaTiedostoLopustaAlkuun()	29
4.3.1.3 lue()	30
4.3.1.4 vapautaMuisti()	30
4.3.1.5 varaaMuistia()	31
4.4 src/linkitettylista.h File Reference	31
4.4.1 Typedef Documentation	32
4.4.1.1 TIEDOT	32
4.4.2 Function Documentation	32
4.4.2.1 kirjoitaTiedostoAlustaLoppuun()	32

4.4.2.2 kirjoitaTiedostoLopustaAlkuun()	32
4.4.2.3 lue()	33
4.4.2.4 vapautaMuisti()	34
4.4.2.5 varaaMuistia()	34
4.5 src/paaohjelma.c File Reference	34
4.5.1 Function Documentation	35
4.5.1.1 main()	35
4.6 src/punamustapuu.c File Reference	36
4.6.1 Function Documentation	37
4.6.1.1 kierraOikealle()	37
4.6.1.2 kierraVasemmalle()	37
4.6.1.3 kirjoitaRB()	38
4.6.1.4 kirjoitaRBTiedostoon()	38
4.6.1.5 korjaaLisays()	38
4.6.1.6 lisaaRBSolmu()	39
4.6.1.7 luoRBPuu()	40
4.6.1.8 vapautaMuistiRB()	40
4.6.1.9 varaaMuistiaRB()	41
4.7 src/yleiset.c File Reference	41
4.7.1 Function Documentation	41
4.7.1.1 binaaripuuValikko()	42
4.7.1.2 kysyNimi()	42
4.7.1.3 listaValikko()	42
4.7.1.4 valikko()	43
4.8 src/yleiset.h File Reference	43
4.8.1 Macro Definition Documentation	43
4.8.1.1 LEN	43
4.8.2 Function Documentation	43
4.8.2.1 binaaripuuValikko()	44
4.8.2.2 kysyNimi()	44
4.8.2.3 listaValikko()	44
4.8.2.4 valikko()	45

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

jono		5
puu		6
RBSolmu		7
tiedot		9

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/ binaaripuu.c	11
src/ binaaripuu.h	11
src/ linkitettylista.c	28
src/ linkitettylista.h	31
src/ paaohjelma.c	34
src/ punamustapuu.c	36
src/ yleiset.c	41
src/ yleiset.h	43

Chapter 3

Class Documentation

3.1 jono Struct Reference

```
#include <binaaripuu.h>
```

Collaboration diagram for jono:

Public Attributes

- [PUU](#) * [pSolmu](#)
- struct [jono](#) * [pSeuraava](#)

3.1.1 Detailed Description

Definition at line 13 of file binaaripuu.h.

3.1.2 Member Data Documentation

3.1.2.1 pSeuraava

```
struct jono* jono::pSeuraava
```

Definition at line 15 of file binaaripuu.h.

3.1.2.2 pSolmu

```
PUU* jono::pSolmu
```

Definition at line 14 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- src/[binaaripuu.h](#)

3.2 puu Struct Reference

```
#include <binaaripuu.h>
```

Collaboration diagram for puu:

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iPituus](#)
- struct [puu](#) * [pOikea](#)
- struct [puu](#) * [pVasen](#)

3.2.1 Detailed Description

Definition at line 5 of file binaaripuu.h.

3.2.2 Member Data Documentation

3.2.2.1 aNimi

```
char puu::aNimi [LEN]
```

Definition at line 6 of file binaaripuu.h.

3.2.2.2 iArvo

```
int puu::iArvo
```

Definition at line 7 of file binaaripuu.h.

3.2.2.3 iPituus

```
int puu::iPituus
```

Definition at line 8 of file binaaripuu.h.

3.2.2.4 pOikea

```
struct puu* puu::pOikea
```

Definition at line 9 of file binaaripuu.h.

3.2.2.5 pVasen

```
struct puu* puu::pVasen
```

Definition at line 10 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- src/[binaaripuu.h](#)

3.3 RBSolmu Struct Reference

```
#include <binaaripuu.h>
```

Collaboration diagram for RBSolmu:

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iVariBitti](#)
- struct [RBSolmu](#) * [pOikea](#)
- struct [RBSolmu](#) * [pVasen](#)
- struct [RBSolmu](#) * [pVanhempi](#)

3.3.1 Detailed Description

Definition at line 18 of file binaaripuu.h.

3.3.2 Member Data Documentation

3.3.2.1 aNimi

```
char RBSolmu::aNimi[LEN]
```

Definition at line 19 of file binaaripuu.h.

3.3.2.2 iArvo

```
int RBSolmu::iArvo
```

Definition at line 20 of file binaaripuu.h.

3.3.2.3 iVariBitti

```
int RBSolmu::iVariBitti
```

Definition at line 21 of file binaaripuu.h.

3.3.2.4 pOikea

```
struct RBSolmu* RBSolmu::pOikea
```

Definition at line 22 of file binaaripuu.h.

3.3.2.5 pVanhempi

```
struct RBSolmu* RBSolmu::pVanhempi
```

Definition at line 24 of file binaaripuu.h.

3.3.2.6 pVasen

```
struct RBSolmu* RBSolmu::pVasen
```

Definition at line 23 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- [src/binaaripuu.h](#)

3.4 tiedot Struct Reference

```
#include <linkitettylista.h>
```

Collaboration diagram for tiedot:

Public Attributes

- char [aSukunimi](#) [[LEN](#)]
- int [iYhteensa](#)
- struct [tiedot](#) * [pSeuraava](#)
- struct [tiedot](#) * [pEdellinen](#)

3.4.1 Detailed Description

Definition at line 5 of file linkitettylista.h.

3.4.2 Member Data Documentation

3.4.2.1 aSukunimi

```
char tiedot::aSukunimi[LEN]
```

Definition at line 6 of file linkitettylista.h.

3.4.2.2 iYhteensa

```
int tiedot::iYhteensa
```

Definition at line 7 of file linkitettylista.h.

3.4.2.3 pEdellinen

```
struct tiedot* tiedot::pEdellinen
```

Definition at line 9 of file linkitettylista.h.

3.4.2.4 pSeuraava

```
struct tiedot* tiedot::pSeuraava
```

Definition at line 8 of file linkitettylista.h.

The documentation for this struct was generated from the following file:

- [src/linkitettylista.h](#)

Chapter 4

File Documentation

4.1 src/binaaripuu.c File Reference

```
#include "binaaripuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for binaaripuu.c:

4.2 src/binaaripuu.h File Reference

```
#include "yleiset.h"
```

Include dependency graph for binaaripuu.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [puu](#)
- struct [jono](#)
- struct [RBSolmu](#)

Typedefs

- typedef struct [puu](#) [PUU](#)
- typedef struct [jono](#) [JONO](#)
- typedef struct [RBSolmu](#) [RBSOLMU](#)

Functions

- **PUU * varaaMuistiaPuulle** (char *pSolmu, int iValiMatka)
Muistin varaaminen puu structin alkioille.
- **PUU * vapautaMuistiPuu** (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- **PUU * lisaaSolmu** (PUU *pAlku, char *pSolmu, int iValiMatka)
Lisaa solmun puuhun.
- **PUU * luoPuu** (char *pNimi, PUU *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- void **kirjoitaBinaaripuu** (char *pNimi, PUU *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- int **kysyArvo** (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsia/poistaa puusta.
- int **syvyyshaku** (char *pKirjoitaTiedostoNimi, PUU *pJuuriSolmu, int iArvo)
- void **leveyshaku** (char *pKirjoitaTiedostoNimi, PUU *pAlku, char *pHaku)
- **JONO * varaaMuistiaJonolle** (PUU *pSolmu)
- **JONO * vapautaMuistiJono** (JONO *pAlku)
- void **kirjoitaTiedostoon** (char *pKirjoitaTiedostoNimi, PUU *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- void **tarkistaLoytyykoSyvyyshaulla** (char *aNimiKirjoitettava, PUU *pJuuriSolmu, int iArvo)
- int **tasapainoitaPuu** (PUU *pAlku)
Palauttaa solmun tasapainokertoimen.
- **PUU * oikeaPuoli** (PUU *pAlku)
Oikean puolen kierto.
- **PUU * vasenPuoli** (PUU *pAlku)
Vasemman puolen kierto.
- int **puunPituus** (PUU *pAlku)
Hakee solmun korkeuden.
- int **suurempiLukuVertailu** (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- **PUU * poistaSolmu** (char *pNimi, int iArvo, PUU *pJuuriSolmu)
- int **binaariHaku** (char *pNimi, PUU *pJuuriSolmu, int iArvo)
- **RBSOLMU * varaaMuistiaRB** (char *pNimi, int iArvo)
Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.
- **RBSOLMU * vapautaMuistiRB** (RBSOLMU *pJuuriSolmu)
- **RBSOLMU * lisaaRBSolmu** (RBSOLMU *pJuurisolmu, RBSOLMU *pUusi)
- **RBSOLMU * luoRBPuu** (RBSOLMU *pJuurisolmu, char *pNimi)
- void **kierraVasemmalle** (RBSOLMU **pJuurisolmu, RBSOLMU *pSolmu)
- void **kierraOikealle** (RBSOLMU **pJuurisolmu, RBSOLMU *pSolmu)
- void **korjaaLisays** (RBSOLMU **pJuurisolmu, RBSOLMU *pUusi)
- void **kirjoitaRBTiedostoon** (char *pNimi, RBSOLMU *pJuurisolmu)
- void **kirjoitaRB** (char *pNimi, RBSOLMU *pJuurisolmu)

4.2.1 Typedef Documentation

4.2.1.1 JONO

```
typedef struct jono JONO
```

4.2.1.2 PUU

```
typedef struct puu PUU
```

4.2.1.3 RBSOLMU

```
typedef struct RBSolmu RBSOLMU
```

4.2.2 Function Documentation

4.2.2.1 binaariHaku()

```
int binaariHaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo )
```

Definition at line 448 of file binaaripuu.c.

```
448                                     {
449
450     if (pJuuriSolmu == NULL) {
451         printf("Puu on tyhjä, luo puurakenne ennen binäärihakua.\n");
452         return (0);
453     }
454
455     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
456
457     if (iArvo == pJuuriSolmu->iArvo) {
458         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
459         return (1);
460     } else if (iArvo < pJuuriSolmu->iArvo) {
461         return binaariHaku(pNimi, pJuuriSolmu->pVasen, iArvo);
462     } else {
463         return binaariHaku(pNimi, pJuuriSolmu->pOikea, iArvo);
464     }
465 }
```

Here is the caller graph for this function:

4.2.2.2 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130 {
131     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuurisolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

Here is the caller graph for this function:

4.2.2.3 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

Here is the caller graph for this function:

4.2.2.4 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 175 of file binaaripuu.c.

```

175                                     {
176     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
177         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
178         return;
179     }*/
180     if (pAlku != NULL) {
181         kirjoitaTiedostoon(pNimi, pAlku);
182         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
183         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
184     }
185     return;
186 }
```

Here is the caller graph for this function:

4.2.2.5 kirjoitaRB()

```

void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```

222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

Here is the caller graph for this function:

4.2.2.6 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```

201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }
```

Here is the caller graph for this function:

4.2.2.7 kirjoitaTiedostoon()

```
void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Kirjoittaa binaaripuun tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 311 of file binaaripuu.c.

```
311                                     {
312     FILE *Tiedosto = NULL;
313
314     if (pAlku == NULL) {
315         return;
316     }
317
318     /* Tiedoston avaaminen. */
319     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
320         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
321         exit(0);
322     }
323     /* Tiedostoon kirjoittaminen. */
324     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
325
326     /* Tiedoston sulkeminen. */
327     fclose(Tiedosto);
328     return;
329 }
```

Here is the caller graph for this function:

4.2.2.8 korjaaLisays()

```
void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )
```

Definition at line 150 of file punamustapuu.c.

```
150                                     {
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
159
160         if (vanhempi == isoVanhempi->pVasen) {
161             seta = isoVanhempi->pOikea;
162             if (seta != NULL && seta->iVariBitti == 0) {
163                 vanhempi->iVariBitti = 1;
164                 seta->iVariBitti = 1;
165                 isoVanhempi->iVariBitti = 0;
166                 pUusi = isoVanhempi;
167             } else {
168                 if (pUusi == vanhempi->pOikea) {
169                     pUusi = vanhempi;
170                     kierraVasemmalle(pJuurisolmu, pUusi);
171                     vanhempi = pUusi->pVanhempi;
172                 }
173                 vanhempi->iVariBitti = 1;
174                 isoVanhempi->iVariBitti = 0;
175                 kierraOikealle(pJuurisolmu, isoVanhempi);
176             }
177         }
178     }
```

```

177
178     } else {
179         seta = isoVanhemp1->pVasen;
180         if (seta != NULL && seta->iVariBitti == 0) {
181             vanhemp1->iVariBitti = 1;
182             seta->iVariBitti = 1;
183             isoVanhemp1->iVariBitti = 0;
184             pUusi = isoVanhemp1;
185         } else {
186             if (pUusi == vanhemp1->pVasen) {
187                 pUusi = vanhemp1;
188                 kierraOikealle(pJuurisolmu, pUusi);
189                 vanhemp1 = pUusi->pVanhemp1;
190             }
191             vanhemp1->iVariBitti = 1;
192             isoVanhemp1->iVariBitti = 0;
193             kierraVasemmalle(pJuurisolmu, isoVanhemp1);
194         }
195     }
196 }
197 (*pJuurisolmu)->iVariBitti = 1;
198 }

```

Here is the caller graph for this function:

4.2.2.9 kysyArvo()

```

int kysyArvo (
    char * pPrompti,
    int iArvo )

```

Kysyy arvon, joka halutaan etsiä/poistaa puusta.

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä

Returns

int Palauttaa kayttajan antaman arvon

Definition at line 195 of file binaaripuu.c.

```

195 {
196     printf("%s", pPrompti);
197     scanf("%d", &iArvo);
198     getchar();
199     return (iArvo);
200 }

```

Here is the caller graph for this function:

4.2.2.10 leveyshaku()

```

void leveyshaku (
    char * pKirjoitaTiedostoNimi,
    PUU * pAlku,
    char * pHaku )

```

Definition at line 229 of file binaaripuu.c.

```

229 {
230     // Leveyshaku nimen mukaan, kirjoitetaan kaydyt solmut tiedostoon.

```

```

231     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
232     JONO *pLoppu = pAlku;
233     JONO *pEdellinen = NULL;
234     int iVertailu = 0;
235     int iLoytyi = 0;
236
237     if (pJuuriSolmu == NULL) {
238         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
239         return;
240     }
241
242     while (pAlku != NULL) {
243         PUU *ptr = pAlku->pSolmu;
244         kirjoitaTiedostoon(pNimi, ptr);
245
246         iVertailu = strcmp(ptr->aNimi, pHaku);
247
248         if (iVertailu == 0) {
249             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
250             iLoytyi = 1;
251             break;
252         }
253
254         if (ptr->pVasen != NULL) {
255             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
256             pLoppu = pLoppu->pSeuraava;
257         }
258
259         if (ptr->pOikea != NULL) {
260             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
261             pLoppu = pLoppu->pSeuraava;
262         }
263
264         pEdellinen = pAlku;
265         pAlku = pAlku->pSeuraava;
266         free(pEdellinen);
267     }
268
269     if (iLoytyi == 0) {
270         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
271     }
272     pAlku = vapautaMuistiJono(pAlku);
273     return;
274 }

```

Here is the caller graph for this function:

4.2.2.11 lisaaRBSolmu()

```

RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )

```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```

38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);

```

```

63         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
64     }
65 }
66 return (pJuurisolmu);
67 }

```

Here is the caller graph for this function:

4.2.2.12 lisaaSolmu()

```

PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iValiMatka )

```

Lisaa solmun puuhun.

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iValiMatka</i>	Solmun arvo.

Returns

PUU*

Definition at line 57 of file binaaripuu.c.

```

57                                     {
58     int iVertailu = 0;
59
60     // Varataan muistia, jos muisti tyhjä.
61     if (pAlku == NULL) {
62         return varaaMuistiaPuulle(pSolmu, iValiMatka);
63     }
64
65     iVertailu = strcmp(pSolmu, pAlku->aNimi);
66
67     // Solmujen lisääminen.
68     if (iValiMatka < pAlku->iArvo) {
69         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iValiMatka);
70     } else if (iValiMatka > pAlku->iArvo) {
71         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iValiMatka);
72     }
73
74     // Testataan, ovatko arvot samat.
75     if (iValiMatka == pAlku->iArvo) {
76         if (iVertailu < 0) {
77             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iValiMatka);
78         } else if (iVertailu > 0) {
79             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iValiMatka);
80         }
81     }
82
83     // Selvitetään paikka, johon solmu lisataan. Tasapainotetaan puu.
84     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
85     int tasapaino = tasapainoitaPuu(pAlku);
86
87     // vasen vasen tasapainotus
88     if ((tasapaino > 1) && (iValiMatka < pAlku->pVasen->iArvo)) {
89         return (oikeaPuoli(pAlku));
90     }
91
92     // vasen oikea tasapainotus
93     if ((tasapaino > 1) && (iValiMatka > pAlku->pVasen->iArvo)) {
94         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
95         return (oikeaPuoli(pAlku));
96     }
97 }

```

```

95     }
96
97     // oikea vasen tasapainotus
98     if ((tasapaino < -1) && (iValiMatka < pAlku->pOikea->iArvo)) {
99         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
100         return (vasenPuoli(pAlku));
101     }
102
103     // oikea oikea tasapainotus
104     if ((tasapaino < -1) && (iValiMatka > pAlku->pOikea->iArvo)) {
105         return (vasenPuoli(pAlku));
106     }
107
108     return (pAlku);
109 }

```

Here is the caller graph for this function:

4.2.2.13 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Luetaan tiedosto ja luodaan puu rakenne.

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 118 of file binaaripuu.c.

```

118
119     FILE *Tiedosto = NULL;
120     char aRivi[LEN] = "";
121     int iOtsikko = 0;
122     char *p1 = NULL, *p2 = NULL;
123     int iValiMatka = 0;
124
125     // Tiedoston avaus
126     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
127         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
128         exit(0);
129     }
130
131     // Tiedoston lukeminen
132     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
133         aRivi[strlen(aRivi) - 1] = '\0';
134
135         // Ohitetaan otsikko
136         if (iOtsikko == 0) {
137             iOtsikko++;
138             continue;
139         }
140
141         // Pilkotaan rivit
142         if ((p1 = strtok(aRivi, ";")) == NULL) {
143             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
144             exit(0);
145         }
146
147         if ((p2 = strtok(NULL, ";")) == NULL) {
148             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");

```

```

149         exit(0);
150     }
151
152     iValiMatka = atoi(p2);
153
154     // Maaritetaan juuri solmu jos sitä ei ole määritetty.
155     if (pJuuriSolmu == NULL) {
156         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iValiMatka);
157         iOtsikko++;
158         continue;
159     }
160
161     pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iValiMatka);
162 }
163
164 // Suljetaan tiedosto
165 fclose(Tiedosto);
166 return (pJuuriSolmu);
167 }

```

Here is the caller graph for this function:

4.2.2.14 luorBPuu()

```

RBSOLMU* luorBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )

```

Definition at line 69 of file punamustapuu.c.

```

69     {
70         FILE *Tiedosto = NULL;
71         char aRivi[LEN] = "";
72         int iOtsikko = 0;
73         char *p1 = NULL, *p2 = NULL;
74         int iArvo = 0;
75
76         if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77             perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78             exit(0);
79         }
80
81         while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82             aRivi[strlen(aRivi) - 1] = '\0';
83             if (iOtsikko == 0) {
84                 iOtsikko++;
85                 continue;
86             }
87
88             if ((p1 = strtok(aRivi, ";")) == NULL) {
89                 perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90                 exit(0);
91             }
92
93             if ((p2 = strtok(NULL, ";")) == NULL) {
94                 perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95                 exit(0);
96             }
97
98             iArvo = atoi(p2);
99
100             RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
101             pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
102             korjaaLisays(&pJuurisolmu, pUusi);
103         }
104
105         fclose(Tiedosto);
106         return (pJuurisolmu);
107     }

```

Here is the caller graph for this function:

4.2.2.15 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Oikean puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 361 of file binaaripuu.c.

```

361     {
362         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
363         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
364             return (pAlku);
365         }
366
367         // Muuttujien ja vakioiden alustaminen
368         PUU *pMuuttuja1 = pAlku->pVasen;
369         PUU *pMuuttuja2 = pMuuttuja1->pOikea;
370
371         // Sijoitetaan arvoja
372         pMuuttuja1->pOikea = pAlku;
373         pAlku->pVasen = pMuuttuja2;
374
375         // Verrataan ja sijoitetaan suuremmat korkeudet
376         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
377         pMuuttuja1->iPituus =
378             suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
379
380         return pMuuttuja1;
381     }

```

Here is the caller graph for this function:

4.2.2.16 poistaSolmu()

```

PUU* poistaSolmu (
    char * pNimi,
    int iArvo,
    PUU * pJuuriSolmu )

```

Definition at line 467 of file binaaripuu.c.

```

467     {
468
469         if (pJuuriSolmu == NULL) {
470             return (pJuuriSolmu);
471         }
472
473         if (iArvo == pJuuriSolmu->iArvo && (strcmp(pNimi, pJuuriSolmu->aNimi) == 0)) {
474             free(pJuuriSolmu);
475             pJuuriSolmu = NULL;
476             printf("Solmu poistettu.\n");
477             return (pJuuriSolmu);
478         } else if (iArvo < pJuuriSolmu->iArvo) {
479             printf("Vertailu: '%s' vs '%s'\n", pNimi, pJuuriSolmu->aNimi);
480             pJuuriSolmu->pVasen = poistaSolmu(pNimi, iArvo, pJuuriSolmu->pVasen);
481         } else if (iArvo > pJuuriSolmu->iArvo) {
482             printf("Vertailu: '%s' vs '%s'\n", pNimi, pJuuriSolmu->aNimi);
483             pJuuriSolmu->pOikea = poistaSolmu(pNimi, iArvo, pJuuriSolmu->pOikea);
484         } else {
485             pJuuriSolmu->pVasen = poistaSolmu(pNimi, iArvo, pJuuriSolmu->pVasen);
486             pJuuriSolmu->pOikea = poistaSolmu(pNimi, iArvo, pJuuriSolmu->pOikea);
487         }
488
489         return (pJuuriSolmu);
490     }

```

Here is the caller graph for this function:

4.2.2.17 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Hakee solmun korkeuden.

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 417 of file binaaripuu.c.

```

417     {
418         // Tarkistetaan, onko Solmu Null
419         if (pAlku == NULL) {
420             return (0);
421         }
422
423         // Palauttaa solmun korkeuden
424         return (pAlku->iPituus);
425     }
```

Here is the caller graph for this function:

4.2.2.18 suurempiLukuVertailu()

```

int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )
```

Vertaa kahta lukua ja palauttaa suuremman.

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 434 of file binaaripuu.c.

```

434     {
435         int iPalautus = 0;
436
437         // Verrataan kumpi luvuista on suurempi.
438         if (iLuku1 > iLuku2) {
439             iPalautus = iLuku1;
440         } else {
441             iPalautus = iLuku2;
442         }
443
444         // Palauttaa suuremman luvun.
445         return (iPalautus);
446     }
```

Here is the caller graph for this function:

4.2.2.19 syvyyshaku()

```

int syvyyshaku (
    char * pKirjoitaTiedostoNimi,
```

```

    PUU * pJuuriSolmu,
    int iArvo )

```

Definition at line 202 of file binaaripuu.c.

```

202                                     {
203     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
204     int iLoytyi = 0;
205
206     if (pAlku == NULL) {
207         printf("Puu on tyhjä, luo puurakenne ennen syvyyshakua.\n");
208         return 0;
209     }
210
211     if (pAlku != NULL) {
212         kirjoitaTiedostoon(pNimi, pAlku);
213
214         if (pAlku->iArvo == iArvo) {
215             iLoytyi = 1;
216             printf("Arvo %d löytyi.\n", iArvo);
217         } else {
218             if (iLoytyi == 0) {
219                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
220             }
221             if (iLoytyi == 0) {
222                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
223             }
224         }
225     }
226     return (iLoytyi);
227 }

```

Here is the caller graph for this function:

4.2.2.20 tarkistaLoytyykoSyvyyshaulalla()

```

void tarkistaLoytyykoSyvyyshaulalla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Definition at line 331 of file binaaripuu.c.

```

331                                     {
332     int iLoytyi = 0;
333     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
334     if (iLoytyi == 0) {
335         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
336     }
337 }

```

Here is the caller graph for this function:

4.2.2.21 tasapainoitaPuu()

```

int tasapainoitaPuu (
    PUU * pAlku )

```

Palauttaa solmun tasapainokertoimen.

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 345 of file binaaripuu.c.

```

345                                     {
346     // Tarkistetaan, onko Null
347     if (pAlku == NULL) {
348         return (0);
349     }
350
351     // Palautetaan tasapainokerroin
352     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
353 }
```

Here is the caller graph for this function:

4.2.2.22 vapautaMuistiJono()

```

JONO* vapautaMuistiJono (
    JONO * pAlku )
```

Definition at line 291 of file binaaripuu.c.

```

291                                     {
292     // Jonon muistin vapauttaminen.
293     JONO *ptr = pAlku;
294     JONO *pSeuraava = NULL;
295
296     while (ptr != NULL) {
297         pSeuraava = ptr->pSeuraava;
298         free(ptr);
299         ptr = pSeuraava;
300     }
301     pAlku = NULL;
302     return (pAlku);
303 }
```

Here is the caller graph for this function:

4.2.2.23 vapautaMuistiPuu()

```

PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Muistin vapauttaminen puu structin alkiosta Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 38 of file binaaripuu.c.

```

38                                     {
39     // Muistin vapauttaminen
40     if (pJuuriSolmu != NULL) {
41         vapautaMuistiPuu(pJuuriSolmu->pVasen);
42         vapautaMuistiPuu(pJuuriSolmu->pOikea);
43         free(pJuuriSolmu);
44     }
```

```

45     pJuuriSolmu = NULL;
46     return (pJuuriSolmu);
47 }

```

Here is the caller graph for this function:

4.2.2.24 vapautaMuistiRB()

```

RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )

```

Definition at line 28 of file punamustapuu.c.

```

28                                     {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB (pJuuriSolmu->pVasen);
31         vapautaMuistiRB (pJuuriSolmu->pOikea);
32         free (pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }

```

Here is the caller graph for this function:

4.2.2.25 varaaMuistiaJonolle()

```

JONO* varaaMuistiaJonolle (
    PUU * pSolmu )

```

Definition at line 276 of file binaaripuu.c.

```

276                                     {
277     JONO *pUusi = NULL;
278
279     // Muistin varaaminen jonolle.
280     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
281         perror("Muistin varaus epäonnistui, lopetetaan");
282         exit(0);
283     }
284
285     pUusi->pSolmu = pSolmu;
286     pUusi->pSeuraava = NULL;
287
288     return (pUusi);
289 }

```

Here is the caller graph for this function:

4.2.2.26 varaaMuistiaPuulle()

```

PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iValiMatka )

```

Muistin varaaminen puu structin alkioille.

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iValiMatka</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 15 of file binaaripuu.c.

```

15                                     {
16     PUU *pUusi = NULL;
17
18     // Muistin varaaminen
19     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
20         perror("Muistin varaus epäonnistui, lopetetaan");
21         exit(0);
22     }
23
24     strcpy(pUusi->aNimi, pSolmu);
25     pUusi->iArvo = iValiMatka;
26     pUusi->iPituus = 1;
27     pUusi->pVasen = NULL;
28     pUusi->pOikea = NULL;
29     return (pUusi);
30 }
```

Here is the caller graph for this function:

4.2.2.27 varaaMuistiaRB()

```

RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```

11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

Here is the caller graph for this function:

4.2.2.28 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )
```

Vasemman puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 389 of file binaaripuu.c.

```

389     {
390         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
391         if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
392             return (pAlku);
393         }
394
395         // Muuttujien ja vakioiden alustaminen
396         PUU *pMuuttujal = pAlku->pOikea;
397         PUU *pMuuttuja2 = pMuuttujal->pVasen;
398
399         // Sijoitetaan arvoja
400         pMuuttujal->pVasen = pAlku;
401         pAlku->pOikea = pMuuttuja2;
402
403         // Verrataan ja sijoitetaan suuremmat korkeudet
404         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
405         pMuuttujal->iPituus =
406             suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
407
408         return pMuuttujal;
409     }

```

Here is the caller graph for this function:

4.3 src/linkitettylista.c File Reference

```

#include "linkitettylista.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Include dependency graph for linkitettylista.c:

Functions

- [TIEDOT * lue](#) (char *pNimi, [TIEDOT *pAlku](#))
- [TIEDOT * varaaMuistia](#) ([TIEDOT *pAlku](#), char *pSukunimi, int iMaara)
- [TIEDOT * vapautaMuisti](#) ([TIEDOT *pAlku](#))
- void [kirjoitaTiedostoAlustaLoppuun](#) (char *pNimi, [TIEDOT *pAlku](#))
- void [kirjoitaTiedostoLopustaAlkuun](#) (char *pNimi, [TIEDOT *pAlku](#))

4.3.1 Function Documentation

4.3.1.1 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Definition at line 98 of file linkitettylista.c.

```
98                                     {
99     FILE *Tiedosto = NULL;
100     TIEDOT *ptr = NULL;
101
102     if (pAlku == NULL) {
103         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
104         return;
105     }
106
107     // Kirjoitus tiedoston avaaminen
108     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
109         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
110         exit(0);
111     }
112
113     // Tiedostoon kirjoittaminen
114     ptr = pAlku;
115     while (ptr != NULL) {
116         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
117         ptr = ptr->pSeuraava;
118     }
119
120     // Tiedoston sulkeminen
121     fclose(Tiedosto);
122     printf("Tiedosto %s kirjoitettu.\n", pNimi);
123     return;
124 }
```

Here is the caller graph for this function:

4.3.1.2 kirjoitaTiedostoLopustaAlkuun()

```
void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Definition at line 126 of file linkitettylista.c.

```
126                                     {
127     FILE *Tiedosto = NULL;
128     TIEDOT *ptr = pAlku;
129
130     if (pAlku == NULL) {
131         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
132         return;
133     }
134
135     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
136         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
137         exit(0);
138     }
139
140     // Mennään listan loppuun.
141     while (ptr->pSeuraava != NULL) {
142         ptr = ptr->pSeuraava;
143     }
144
145     // Lopusta alkuun, kirjoitetaan tiedostoon.
146     while (ptr != NULL) {
147         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148         ptr = ptr->pEdellinen;
149     }
150
151     fclose(Tiedosto);
152     printf("Tiedosto %s kirjoitettu.\n", pNimi);
153     return;
154 }
```

Here is the caller graph for this function:

4.3.1.3 lue()

```
TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Definition at line 8 of file linkitettylista.c.

```
8
9     FILE *Tiedosto = NULL;
10    char aRivi[LEN] = "";
11    char *p1 = NULL, *p2 = NULL;
12    int iOtsikko = 0;
13    int iMaara = 0;
14
15    /* Tiedoston avaaminen. */
16
17    if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
18        perror("Tiedoston avaaminen epäonnistui, lopetetaan");
19        exit(0);
20    }
21
22    /* Tiedoston lukeminen */
23
24    while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
25
26        aRivi[strlen(aRivi) - 1] = '\0';
27
28        if (iOtsikko == 0) {
29            iOtsikko = 1;
30            continue;
31        }
32
33        if ((p1 = strtok(aRivi, ";")) == NULL) {
34            perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
35            exit(0);
36        }
37
38        if ((p2 = strtok(NULL, ";")) == NULL) {
39            perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
40            exit(0);
41        }
42
43        iMaara = atoi(p2);
44
45        /* Muistin varaaminen ja linkitetyn listan luominen. */
46
47        pAlku = varaaMuistia(pAlku, p1, iMaara);
48    }
49    /* Tiedoston sulkeminen. */
50    fclose(Tiedosto);
51    printf("Tiedosto '%s' luettu.\n", pNimi);
52    return (pAlku);
53 }
```

Here is the caller graph for this function:

4.3.1.4 vapautaMuisti()

```
TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Definition at line 86 of file linkitettylista.c.

```
86
87    /* Muistin vapauttaminen. */
88    TIEDOT *ptr = pAlku;
89    while (ptr != NULL) {
90        pAlku = ptr->pSeuraava;
91        free(ptr);
92        ptr = pAlku;
93    }
94    pAlku = NULL;
95    return (pAlku);
96 }
```

Here is the caller graph for this function:

4.3.1.5 varaaMuistia()

```
TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pSukunimi,
    int iMaara )
```

Definition at line 55 of file linkitettylista.c.

```
55
56     TIEDOT *pUusi = NULL;
57     TIEDOT *ptr = NULL;
58
59     /* Muistin varaaminen. */
60
61     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
62         perror("Muistin varaus epäonnistui, lopetetaan");
63         exit(0);
64     }
65
66     /* Kaksisuuntaisen linkitetyn listan luominen. */
67
68     strcpy(pUusi->aSukunimi, pSukunimi);
69     pUusi->iYhteensa = iMaara;
70     pUusi->pSeuraava = NULL;
71
72     if (pAlku == NULL) {
73         pUusi->pEdellinen = NULL;
74         pAlku = pUusi;
75     } else {
76         ptr = pAlku;
77         while (ptr->pSeuraava != NULL) {
78             ptr = ptr->pSeuraava;
79         }
80         ptr->pSeuraava = pUusi;
81         pUusi->pEdellinen = ptr;
82     }
83     return (pAlku);
84 }
```

Here is the caller graph for this function:

4.4 src/linkitettylista.h File Reference

```
#include "yleiset.h"
```

Include dependency graph for linkitettylista.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [tiedot](#)

Typedefs

- typedef struct [tiedot](#) [TIEDOT](#)

Functions

- [TIEDOT * lue](#) (char *pNimi, [TIEDOT](#) *pAlku)
- [TIEDOT * varaaMuistia](#) ([TIEDOT](#) *pAlku, char *pSukunimi, int iMaara)
- [TIEDOT * vapautaMuisti](#) ([TIEDOT](#) *pAlku)
- void [kirjoitaTiedostoAlustaLoppuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT](#) *pAlku)
- void [kirjoitaTiedostoLopustaAlkuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT](#) *pAlku)

4.4.1 Typedef Documentation

4.4.1.1 TIEDOT

```
typedef struct tiedot TIEDOT
```

4.4.2 Function Documentation

4.4.2.1 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pKirjoitaTiedostoNimi,
    TIEDOT * pAlku )
```

Definition at line 98 of file linkitettylista.c.

```

98
99     FILE *Tiedosto = NULL;
100     TIEDOT *ptr = NULL;
101
102     if (pAlku == NULL) {
103         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
104         return;
105     }
106
107     // Kirjoitus tiedoston avaaminen
108     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
109         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
110         exit(0);
111     }
112
113     // Tiedostoon kirjoittaminen
114     ptr = pAlku;
115     while (ptr != NULL) {
116         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
117         ptr = ptr->pSeuraava;
118     }
119
120     // Tiedoston sulkeminen
121     fclose(Tiedosto);
122     printf("Tiedosto %s kirjoitettu.\n", pNimi);
123     return;
124 }
```

Here is the caller graph for this function:

4.4.2.2 kirjoitaTiedostoLopustaAlkuun()

```
void kirjoitaTiedostoLopustaAlkuun (
    char * pKirjoitaTiedostoNimi,
    TIEDOT * pAlku )
```

Definition at line 126 of file linkitettylista.c.

```

126
127     FILE *Tiedosto = NULL;
128     TIEDOT *ptr = pAlku;
129
130     if (pAlku == NULL) {
131         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
```

```

132     return;
133 }
134
135 if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
136     perror("Tiedoston avaaminen epäonnistui, lopetetaan");
137     exit(0);
138 }
139
140 // Mennään listan loppuun.
141 while (ptr->pSeuraava != NULL) {
142     ptr = ptr->pSeuraava;
143 }
144
145 // Lopusta alkuun, kirjoitetaan tiedostoon.
146 while (ptr != NULL) {
147     fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148     ptr = ptr->pEdellinen;
149 }
150
151 fclose(Tiedosto);
152 printf("Tiedosto %s kirjoitettu.\n", pNimi);
153 return;
154 }

```

Here is the caller graph for this function:

4.4.2.3 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )

```

Definition at line 8 of file linkitettylista.c.

```

8 {
9     FILE *Tiedosto = NULL;
10     char aRivi[LEN] = "";
11     char *p1 = NULL, *p2 = NULL;
12     int iOtsikko = 0;
13     int iMaara = 0;
14
15     /* Tiedoston avaaminen. */
16
17     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
18         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
19         exit(0);
20     }
21
22     /* Tiedoston lukeminen */
23
24     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
25
26         aRivi[strlen(aRivi) - 1] = '\0';
27
28         if (iOtsikko == 0) {
29             iOtsikko = 1;
30             continue;
31         }
32
33         if ((p1 = strtok(aRivi, ";")) == NULL) {
34             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
35             exit(0);
36         }
37
38         if ((p2 = strtok(NULL, ";")) == NULL) {
39             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
40             exit(0);
41         }
42
43         iMaara = atoi(p2);
44
45         /* Muistin varaaminen ja linkitetyn listan luominen. */
46         pAlku = varaaMuistia(pAlku, p1, iMaara);
47     }
48
49     /* Tiedoston sulkeminen. */
50     fclose(Tiedosto);
51     printf("Tiedosto '%s' luettu.\n", pNimi);
52     return (pAlku);
53 }

```

Here is the caller graph for this function:

4.4.2.4 vapautaMuisti()

```
TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Definition at line 86 of file linkitettylista.c.

```
86 {
87     /* Muistin vapauttaminen. */
88     TIEDOT *ptr = pAlku;
89     while (ptr != NULL) {
90         pAlku = ptr->pSeuraava;
91         free(ptr);
92         ptr = pAlku;
93     }
94     pAlku = NULL;
95     return (pAlku);
96 }
```

Here is the caller graph for this function:

4.4.2.5 varaaMuistia()

```
TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pSukunimi,
    int iMaara )
```

Definition at line 55 of file linkitettylista.c.

```
55 {
56     TIEDOT *pUusi = NULL;
57     TIEDOT *ptr = NULL;
58
59     /* Muistin varaaminen. */
60
61     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
62         perror("Muistin varaus epäonnistui, lopetetaan");
63         exit(0);
64     }
65
66     /* Kaksisuuntaisen linkitetyn listan luominen. */
67
68     strcpy(pUusi->aSukunimi, pSukunimi);
69     pUusi->iYhteensa = iMaara;
70     pUusi->pSeuraava = NULL;
71
72     if (pAlku == NULL) {
73         pUusi->pEdellinen = NULL;
74         pAlku = pUusi;
75     } else {
76         ptr = pAlku;
77         while (ptr->pSeuraava != NULL) {
78             ptr = ptr->pSeuraava;
79         }
80         ptr->pSeuraava = pUusi;
81         pUusi->pEdellinen = ptr;
82     }
83     return (pAlku);
84 }
```

Here is the caller graph for this function:

4.5 src/paaohjelma.c File Reference

```
#include "binaaripuu.h"
#include "linkitettylista.h"
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
Include dependency graph for paaohjelma.c:
```

Functions

- int [main](#) (void)
13.3.

4.5.1 Function Documentation

4.5.1.1 main()

```
int main (
    void )
```

13.3.

: Main ei enää tee null checkkejä. Johtaa siihen, että kirjoitettavien tiedostojen nimet kysytään turhaan. Onko tämä kuitenkin selkeämpi?

Syvyyshaku siistittiin [binaaripuu.c](#) sijaitsevalla helper-aliohjelmalla.

Definition at line 15 of file paaohjelma.c.

```
15 {
16     char aNimiLuettava[LEN] = "";
17     char aNimiKirjoitettava[LEN] = "";
18     char aHaettavaNimi[LEN] = "";
19     TIEDOT *pAlku = NULL;
20     PUU *pJuuriSolmu = NULL;
21     RBSOLMU *pRBJuuri = NULL;
22     int iValinta = 0;
23     int iValinta2 = 0;
24     int iArvo = 0;
25
26     do {
27         iValinta = valikko();
28         if (iValinta == 1) {
29             // Lista:
30             do {
31                 iValinta2 = listaValikko();
32                 if (iValinta2 == 1) {
33                     kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
34                     pAlku = lue(aNimiLuettava, pAlku);
35                 } else if (iValinta2 == 2) {
36                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
37                     kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
38                 } else if (iValinta2 == 3) {
39                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
40                     kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
41                 } else if (iValinta2 == 4) {
42                     pAlku = vapautaMuisti(pAlku);
43                     printf("Muisti vapautettu.\n");
44                 } else if (iValinta2 == 0) {
45                     printf("Palataan takaisin päävalikkoon.\n");
46                 } else {
47                     printf("Tunteamaton valinta, yritä uudestaan.\n");
48                 }
49             } while (iValinta2 != 0);
50
51             /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
52              * käyttäjä ei muista sitä tehdä. */
53             pAlku = vapautaMuisti(pAlku);
54         } else if (iValinta == 2) {
55             // Binaaripuu
56             do {
57                 iValinta2 = binaaripuuValikko();
58                 if (iValinta2 == 1) {
59                     kysyNimi("Tiedoston nimi: ", aNimiLuettava);
60                     pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
61                 } else if (iValinta2 == 2) {
62                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
63                     kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
```

```

64         } else if (iValinta2 == 3) {
65             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
66             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
67             tarkistaLoytyykoSyvyyshaulalla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
68         } else if (iValinta2 == 4) {
69             kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
70             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
71             leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
72         } else if (iValinta2 == 5) {
73             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
74             printf("Muisti vapautettu.\n");
75         } else if (iValinta2 == 6) {
76             kysyNimi("Anna poistettava nimi: ", aHaettavaNimi);
77             kysyArvo("Anna poistettavan nimen arvo: ", iArvo);
78             pJuuriSolmu = poistaSolmu(aHaettavaNimi, iArvo, pJuuriSolmu);
79         } else if (iValinta2 == 7) {
80             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
81             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
82             binaariHaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
83         } else if (iValinta2 == 8) {
84             // Aino säätää ja testaa
85             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
86             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
87             pRBJuuri = luorBPuu(pRBJuuri, aNimiLuettava);
88             kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
89             pRBJuuri = vapautaMuistiRB(pRBJuuri);
90         } else if (iValinta2 == 0) {
91             printf("Palataan takaisin päävalikkoon.\n");
92         } else {
93             printf("Tunteaton valinta, yritä uudestaan.\n");
94         }
95     } while (iValinta2 != 0);
96
97     /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
98     * käyttäjä ei muista sitä tehdä. */
99     pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
100 } else if (iValinta == 0) {
101     printf("Lopetetaan.\n");
102 } else {
103     printf("Tunteaton valinta, yritä uudestaan.\n");
104 }
105 printf("\n");
106 } while (iValinta != 0);
107
108 printf("Kiitos ohjelman käytöstä.\n");
109 return (0);
110 }

```

4.6 src/punamustapuu.c File Reference

```

#include "binaaripuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Include dependency graph for punamustapuu.c:

Functions

- [RBSOLMU * varaaMuistiaRB](#) (char *pNimi, int iArvo)
L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmia, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmia muokattuina.
- [RBSOLMU * vapautaMuistiRB](#) (RBSOLMU *pJuuriSolmu)
- [RBSOLMU * lisaaRBSolmu](#) (RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi)
- [RBSOLMU * luorBPuu](#) (RBSOLMU *pJuuriSolmu, char *pNimi)
- void [kierraVasemmalle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [kierraOikealle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [korjaaLisays](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pUusi)
- void [kirjoitaRBTiedostoon](#) (char *pNimi, RBSOLMU *pJuuriSolmu)
- void [kirjoitaRB](#) (char *pNimi, RBSOLMU *pJuuriSolmu)

4.6.1 Function Documentation

4.6.1.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130 {
131     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuurisolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

Here is the caller graph for this function:

4.6.1.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

Here is the caller graph for this function:

4.6.1.3 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```
222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

Here is the caller graph for this function:

4.6.1.4 kirjoitaRBTiedostoon()

```
void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```
201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }
```

Here is the caller graph for this function:

4.6.1.5 korjaaLisays()

```
void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )
```

Definition at line 150 of file punamustapuu.c.

```
150                                     {
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
```

```

159
160     if (vanhemp1 == isoVanhemp1->pVasen) {
161         seta = isoVanhemp1->pOikea;
162         if (seta != NULL && seta->iVariBitti == 0) {
163             vanhemp1->iVariBitti = 1;
164             seta->iVariBitti = 1;
165             isoVanhemp1->iVariBitti = 0;
166             pUusi = isoVanhemp1;
167         } else {
168             if (pUusi == vanhemp1->pOikea) {
169                 pUusi = vanhemp1;
170                 kierraVasemmalle(pJuurisolmu, pUusi);
171                 vanhemp1 = pUusi->pVanhemp1;
172             }
173             vanhemp1->iVariBitti = 1;
174             isoVanhemp1->iVariBitti = 0;
175             kierraOikealle(pJuurisolmu, isoVanhemp1);
176         }
177     }
178     } else {
179         seta = isoVanhemp1->pVasen;
180         if (seta != NULL && seta->iVariBitti == 0) {
181             vanhemp1->iVariBitti = 1;
182             seta->iVariBitti = 1;
183             isoVanhemp1->iVariBitti = 0;
184             pUusi = isoVanhemp1;
185         } else {
186             if (pUusi == vanhemp1->pVasen) {
187                 pUusi = vanhemp1;
188                 kierraOikealle(pJuurisolmu, pUusi);
189                 vanhemp1 = pUusi->pVanhemp1;
190             }
191             vanhemp1->iVariBitti = 1;
192             isoVanhemp1->iVariBitti = 0;
193             kierraVasemmalle(pJuurisolmu, isoVanhemp1);
194         }
195     }
196 }
197 (*pJuurisolmu)->iVariBitti = 1;
198 }

```

Here is the caller graph for this function:

4.6.1.6 lisaaRBSolmu()

```

RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )

```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```

38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhemp1 = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhemp1 = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhemp1 = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhemp1 = pJuurisolmu;
62         }
63     }
64     }
65     return (pJuurisolmu);
66 }
67 }

```

Here is the caller graph for this function:

4.6.1.7 luoRBPuu()

```
RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )
```

Definition at line 69 of file punamustapuu.c.

```
69 {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {
89             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90             exit(0);
91         }
92
93         if ((p2 = strtok(NULL, ";")) == NULL) {
94             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95             exit(0);
96         }
97
98         iArvo = atoi(p2);
99
100         RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
101         pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
102         korjaaLisays(&pJuurisolmu, pUusi);
103     }
104
105     fclose(Tiedosto);
106     return (pJuurisolmu);
107 }
```

Here is the caller graph for this function:

4.6.1.8 vapautaMuistiRB()

```
RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )
```

Definition at line 28 of file punamustapuu.c.

```
28 {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB(pJuuriSolmu->pVasen);
31         vapautaMuistiRB(pJuuriSolmu->pOikea);
32         free(pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }
```

Here is the caller graph for this function:

4.6.1.9 varaaMuistiaRB()

```
RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmat, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmia muokattuina.

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```
11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

Here is the caller graph for this function:

4.7 src/yleiset.c File Reference

```
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
Include dependency graph for yleiset.c:
```

Functions

- int [valikko](#) (void)
- int [listaValikko](#) (void)
- int [binaaripuuValikko](#) (void)
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)

4.7.1 Function Documentation

4.7.1.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Definition at line 34 of file yleiset.c.

```
34     {
35         int iValinta = 0;
36         printf("\nValitse haluamasi toiminto:\n");
37         printf("1) Luo puu\n");
38         printf("2) Kirjoita AVL puu tiedostoon\n");
39         printf("3) Syvyyshaku\n");
40         printf("4) Leveyshaku\n");
41         printf("5) Tyhjennä puu\n");
42         printf("6) Poista solmu\n");
43         printf("7) Binäärihaku AVL puusta\n");
44         printf("8) Punamustapuu\n");
45         printf("0) Takaisin\n");
46         printf("Anna valintasi: ");
47         scanf("%d", &iValinta);
48         getchar();
49         return iValinta;
50 }
```

Here is the caller graph for this function:

4.7.1.2 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Definition at line 52 of file yleiset.c.

```
52     {
53         printf("%s", pPrompti);
54         scanf("%s", pNimi);
55         getchar();
56         return (pNimi);
57 }
```

Here is the caller graph for this function:

4.7.1.3 listaValikko()

```
int listaValikko (
    void )
```

Definition at line 20 of file yleiset.c.

```
20     {
21         int iValinta = 0;
22         printf("\nValitse haluamasi toiminto:\n");
23         printf("1) Lue tiedosto\n");
24         printf("2) Kirjoita tiedosto alusta loppuun\n");
25         printf("3) Kirjoita tiedosto lopusta alkuun\n");
26         printf("4) Tyhjennä taulukko\n");
27         printf("0) Takaisin\n");
28         printf("Anna valintasi: ");
29         scanf("%d", &iValinta);
30         getchar();
31         return iValinta;
32 }
```

Here is the caller graph for this function:

4.7.1.4 valikko()

```
int valikko (
    void )
```

Definition at line 8 of file yleiset.c.

```
8      {
9      int iValinta = 0;
10     printf("Valitse haluamasi toiminto:\n");
11     printf("1) Lista\n");
12     printf("2) Binääripuu\n");
13     printf("0) Lopeta\n");
14     printf("Anna valintasi: ");
15     scanf("%d", &iValinta);
16     getchar();
17     return iValinta;
18 }
```

Here is the caller graph for this function:

4.8 src/yleiset.h File Reference

This graph shows which files directly or indirectly include this file:

Macros

- #define [LEN](#) 50

Functions

- int [valikko](#) (void)
- int [listaValikko](#) (void)
- int [binaaripuuValikko](#) (void)
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)

4.8.1 Macro Definition Documentation

4.8.1.1 LEN

```
#define LEN 50
```

Definition at line 4 of file yleiset.h.

4.8.2 Function Documentation

4.8.2.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Definition at line 34 of file yleiset.c.

```
34     {
35         int iValinta = 0;
36         printf("\nValitse haluamasi toiminto:\n");
37         printf("1) Luo puu\n");
38         printf("2) Kirjoita AVL puu tiedostoon\n");
39         printf("3) Syvyyshaku\n");
40         printf("4) Leveyshaku\n");
41         printf("5) Tyhjennä puu\n");
42         printf("6) Poista solmu\n");
43         printf("7) Binäärihaku AVL puusta\n");
44         printf("8) Punamustapuu\n");
45         printf("0) Takaisin\n");
46         printf("Anna valintasi: ");
47         scanf("%d", &iValinta);
48         getchar();
49         return iValinta;
50 }
```

Here is the caller graph for this function:

4.8.2.2 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Definition at line 52 of file yleiset.c.

```
52     {
53         printf("%s", pPrompti);
54         scanf("%s", pNimi);
55         getchar();
56         return (pNimi);
57 }
```

Here is the caller graph for this function:

4.8.2.3 listaValikko()

```
int listaValikko (
    void )
```

Definition at line 20 of file yleiset.c.

```
20     {
21         int iValinta = 0;
22         printf("\nValitse haluamasi toiminto:\n");
23         printf("1) Lue tiedosto\n");
24         printf("2) Kirjoita tiedosto alusta loppuun\n");
25         printf("3) Kirjoita tiedosto lopusta alkuun\n");
26         printf("4) Tyhjennä taulukko\n");
27         printf("0) Takaisin\n");
28         printf("Anna valintasi: ");
29         scanf("%d", &iValinta);
30         getchar();
31         return iValinta;
32 }
```

Here is the caller graph for this function:

4.8.2.4 valikko()

```
int valikko (
    void )
```

Definition at line 8 of file yleiset.c.

```
8      {
9      int iValinta = 0;
10     printf("Valitse haluamasi toiminto:\n");
11     printf("1) Lista\n");
12     printf("2) Binääripuu\n");
13     printf("0) Lopeta\n");
14     printf("Anna valintasi: ");
15     scanf("%d", &iValinta);
16     getchar();
17     return iValinta;
18 }
```

Here is the caller graph for this function:

